Sprint 1 — MVP

Tasks

Infrastructure

- Create the monorepository structure
- Configure GitHub Actions (lint, build, test)
- Prepare a Dockerfile for Incident Service
- Prepare a Dockerfile for Report Service
- Configure Docker Compose
- Set up PostgreSQL
- Configure environment variables and local development environment

Data Model and API

- Design the database schema for incidents
- Design the database schema for incident events
- Create SQL migrations
- Design the OpenAPI contract for Report Service
- Design DTOs for communication between Incident Service and Report Service
- Prepare UML and architecture diagrams

Incident Service (Go)

- Implement incident domain models
- Implement the incident repository
- Implement the incident event repository
- Implement incident creation
- Implement adding events to the timeline
- Implement retrieval of the active incident by chat_id
- Implement retrieval of the incident timeline
- Implement incident severity updates
- Implement incident closure
- Implement an HTTP client for calling Report Service

Telegram Bot

- Configure integration with the Telegram Bot API
- Implement Long Polling
- Implement handling of the `/incident create` command
- Implement handling of the `/incident <message>` command
- Implement handling of the `/timeline` command
- Implement handling of the `/incident close` command
- Implement inline buttons for incident management
- Implement severity updates via buttons
- Implement timeline message formatting
- Implement sending PDF reports to the chat

Report Service (Python)

- Implement the report generation endpoint
- Implement input data validation
- Implement PDF file generation
- Implement the incident overview section
- Implement the incident participants section
- Implement the timeline section
- Implement the executive summary section
- Return the generated PDF via HTTP response

Documentation

- Prepare the system architecture description
- Prepare the database schema documentation
- Prepare the OpenAPI specification
- Prepare Telegram command documentation
- Prepare edge case documentation
- Prepare user stories and use cases

Demo and QA

- Test incident creation
- Test adding events to the timeline
- Test severity updates
- Test incident closure
- Test PDF report generation
- Conduct end-to-end testing of the complete workflow

- Fix critical issues
- Prepare a demo scenario for the final presentation

Incident Lifecycle

Creating an Incident

An incident is created using the command:

```
/incident create <description>
```

Example:

```
/incident create Payment Service Down
```

After creation, the bot sends a message:

```
🚨 Incident created

Title:
Payment Service Down

Severity:
MEDIUM

[Change Severity]
[Show Timeline]
[Close Incident]
```

The bot provides inline buttons for:

- Viewing the current incident timeline
- Closing the incident
- Changing severity by choosing it from the pop up message:

```
Select severity:
🟢 LOW
🟡 MEDIUM
🟠 HIGH
```

Also actions are available through commands:

```
/timeline
/incident close
```

Adding Events to the Timeline

To add important investigation updates, team members use:

```
/incident <message>
```

Examples:

```
/incident Started investigating database issues

/incident Found a bug in Redis configuration

/incident Restarted payment service
```

Each command creates a new timeline event linked to the active incident.

Viewing the Timeline

The current timeline can be viewed at any time using:

```
/timeline
```

The bot displays all recorded events in chronological order.

Format:

```
📋 Timeline

[14:03] 🚨 Incident created

[14:05] Ролан
Started investigating database issues

[14:09] Богдан
Found connection pool errors

[14:12] Богдан
Restarted payment service
```

```
[14:18] ✅ Incident closed
```

Closing an Incident

To close an active incident:

```
/incident close
```

The bot responds:

```
Incident closed

Duration: 15 min

Generating report...
```

The Go service collects incident data and requests report generation from the Report Service.

Report Generation

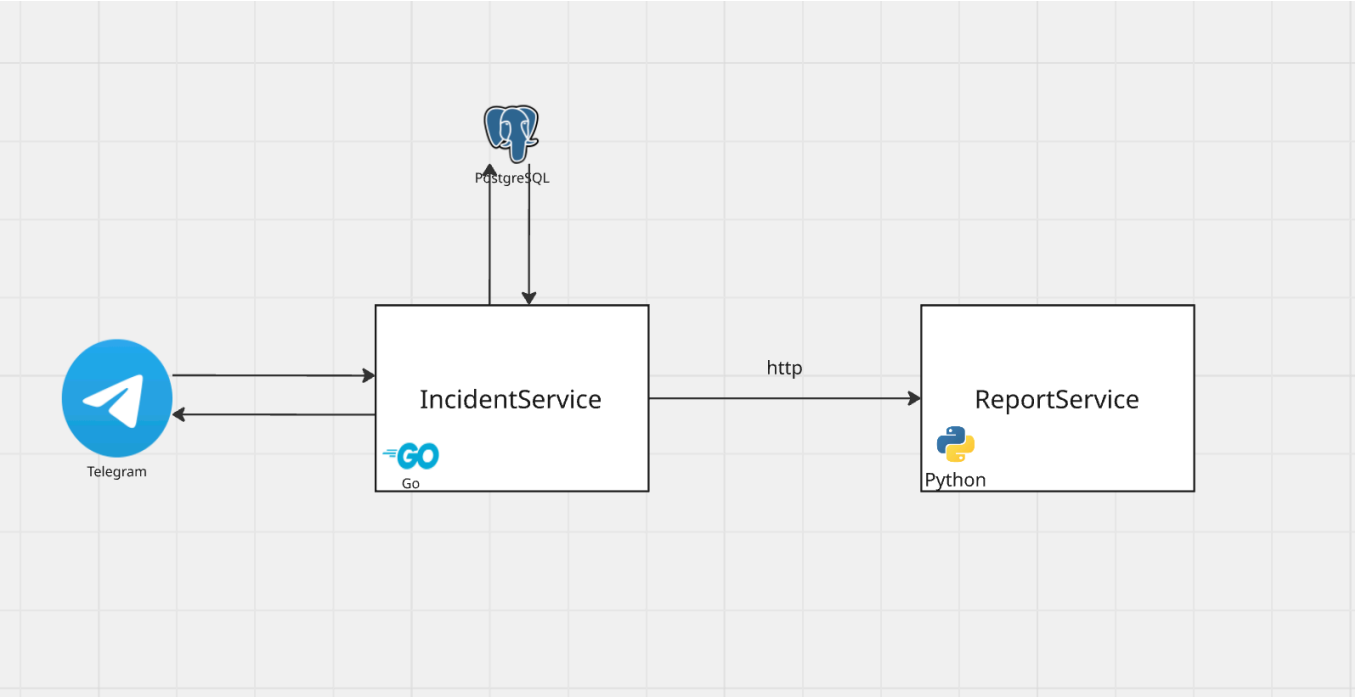
After the report is generated, the bot sends:

```
✅ Incident #15 closed

Duration: 15 min
Participants: 3
Timeline events: 5

The incident report has been generated and attached below.
```

The generated PDF report is attached to the message and contains:

- Incident metadata
- Participants
- Timeline of events

Технический Scope Sprint 1



1. Implement the Go-Based Incident Management Service

Responsibilities:

- Integration with the Telegram Bot API (Long Polling)
- Incident lifecycle management
- PostgreSQL integration
- REST API implementation
- Integration with the report generation service

Functionality:

- Create an incident (`/incident create`)
- Update incident status
- Add messages and comments to an incident
- Close an incident
- Retrieve the incident timeline
- Store incidents and events in PostgreSQL
- Build timelines based on stored events
- Retrieve the list of participants using `DISTINCT user_id` from `incident_events`
- Request report generation after incident closure
- Send PDF reports to Telegram

Database:

- `incidents`

- `incident_events`

2. Implement the Python-Based Report Generation Service

Responsibilities:

- Receive incident data from the Go service
- Generate PDF reports
- Return generated PDFs to the Go service

Functionality:

- Generate an executive summary of the incident
- Include general incident information
- Include the list of participants
- Include the complete event timeline
- Return the PDF via HTTP (`application/pdf`)

3. Service Interaction

Communication Method:

- HTTP REST

Workflow:
`Telegram → Go Service → PostgreSQL`

After the command:
`/incident close`

The Go service:

- retrieves incident data from the database
- creates a report DTO
- sends an HTTP request to the Python service

The Python service:

- generates a PDF report
- returns the PDF file in the response

The Go service:

- receives the PDF
- sends the document to the Telegram chat

4. Infrastructure

- Create a monorepository
- Prepare a Dockerfile for each service
- Configure Docker Compose for local development
- Set up PostgreSQL
- Create initial database migrations
- Configure GitHub Actions (build, lint, test)
- Deploy the bot to a dedicated server

5. Sprint Outcome

The following end-to-end user workflow must be fully operational:

```
/incident create
↓
add messages via /incident <message>
↓
/incident close
↓
PDF report generation
↓
report delivery to Telegram
```

Database Schema

Table: incidents

- id (UUID)
- chat_id (Telegram chat ID)
- title
- status (ACTIVE, CLOSED)
- severity (LOW, MEDIUM, HIGH)
- created_by (Telegram user ID)
- created_at
- closed_at

Table: incident_events

- id (UUID)
- incident_id (FK -> incidents.id)
- event_type (INCIDENT_CREATED, COMMENT_ADDED, INCIDENT_CLOSED)

- author_id (Telegram user ID)
- username
- message
- created_at

Поиск участников инцидента:

```
SELECT DISTINCT user_id, username
FROM incident_events
WHERE incident_id = ?
```

API contracts

Final version can vary depending on the implementation

Generate Report

```
POST /api/v1/reports/generate
```

Request:

```
{
  "incident": {
    "id": "uuid",
    "title": "Payment Service Down",
    "createdAt": "2026-06-01T14:03:00Z",
    "closedAt": "2026-06-01T14:18:00Z"
  },
  "participants": [
    {
      "userId": 1,
      "username": "rolan"
    }
  ],
  "timeline": [
    {
      "timestamp": "2026-06-01T14:05:00Z",
      "username": "rolan",
      "message": "Started investigating database issues"
    }
  ]
}
```

Response:

```
200 OK
Content-Type: application/pdf
```

- Body: PDF bytes

```
./
├── incident-service/..
│
├── report-service/..
│
├── docs/..
│
├── README.md
├── LICENSE.md
└── docker-compose.yml
```